1 Order dependence in `pangraph build`

Branch `debug/order-dependence`*. Investigation of the report in* `tmp/pangraph_order_bug/`*.*

1.1 Summary

`pangraph build` produces a materially different graph depending only on the order the input FASTA files are listed on the command line — 452 to 509 blocks for the same three genomes, a ~ 12% spread. The reported hypothesis was that graph merging follows argument order rather than the guide tree. That is *not* what happens: the guide tree is honoured exactly.

The real cause is that block identifiers are assigned from the input position, those identifiers are handed to `minimap2` as sequence names, and `minimap2` is invoked with `-X`, which keeps only one direction of each pairwise alignment — chosen by *string comparison of those names*. Argument order therefore decides, for every pair of blocks, which one is the query and which one is the reference. That choice is not neutral: the alignment is asymmetric, and downstream the reference is privileged when choosing the consensus that the merged block inherits.

The proposed fix is to make the aligner boundary *canonical*: order and name blocks by a hash of their consensus sequence, so that everything the aligner sees is a pure function of the block content, independent of input order, tree order and identifier assignment.

1.2 Evidence

1.2.1 The guide tree is obeyed

Running orders `ERS990151 AP025961 NZ_CP035927` and `NZ_CP035927 ERS990151 AP025961` with the same `--guide-tree` yields identical logged topology and an identical merge schedule:

```
Guide tree (newick): ((ERS990151,AP025961),NZ_CP035927);
=== Graph merging start:     clades sizes 1 + 1
=== Graph merging completed: clades sizes 1 + 1 -> 2
=== Graph merging start:     clades sizes 2 + 1
=== Graph merging completed: clades sizes 2 + 1 -> 3
```

`build_tree_from_newick` (`tree/newick.rs:70`) attaches graphs to leaves by name through a `BTreeMap`, so topology and left/right assignment are fixed entirely by the tree file. Merge scheduling is not the culprit.

1.2.2 Order dependence survives when merge order cannot vary

With two genomes and the tree `(ERS990151,AP025961)` there is exactly one possible merge. The result still depends on argument order:

argument order	blocks	consensus shared with other order	total consensus bp
`ERS990151 AP025961`	62	37 / 62	3 358 923
`AP025961 ERS990151`	62	37 / 62	3 358 784

Same block count, same depth multiset, but 25 of 62 blocks carry a different consensus sequence and the total consensus length differs by 139 bp. This rules out merge scheduling conclusively.

1.2.3 The alignment direction flips

A probe calling `align_with_minimap2_lib` directly on the two genomes, varying only the `BlockId` assignment, gives:

ids	hits	cross-genome direction (qry → ref)	matched bp
0, 1	217	`ERS990151 -> AP025961` × 147	263 139

ids	hits	cross-genome direction (qry → ref)	matched bp
1, 0	216	`AP025961 -> ERS990151` × 146	266 047
1, 2	217	`ERS990151 -> AP025961` × 147	263 139
2, 10	216	`AP025961 -> ERS990151` × 146	266 047

Three things follow. The cross-genome hits flip direction wholesale. The hit sets genuinely differ (147 vs 146 hits, ~ 2 900 matched bases), so this is not a relabelling. And only the *relative* order matters — `0,1` behaves identically to `1,2`, while `2,10` behaves like `1,0`, proving the comparison is lexicographic on the decimal string rather than numeric, since `"10" < "2"`.

1.3 Root cause

A four-step chain from argument position to alignment direction.

1. **Identifiers are the input position.** `Pangraph::singleton` (`pangraph/pangraph.rs:29`) sets `NodeId(fasta.index)`, `BlockId(fasta.index)` and `PathId(fasta.index)`, where `fasta.index` is the record's position in the concatenated input.
2. **Identifiers become aligner sequence names.** `align_with_minimap2_lib` (`align/minimap2_lib/align_with_minimap2_lib.rs:19`) stringifies the `BlockId` as the name passed to the index.
3. **`minimap2` picks the direction by string comparison.** The call sets `X: true` (same file, line 55), which raises `MM_F_NO_DUAL`. In the vendored source, `skip_seed` (`packages/minimap2-sys/minimap2/map.c:89`) drops every hit where `strcmp(qname, tname) > 0`; the header documents the flag as *"skip pairs where query name is lexicographically larger than target name"* (`minimap.h:11`). Each pair is aligned in one direction only.
4. **Query and reference are not interchangeable.** `assign_anchor_block` (`pangraph/reweave.rs:144`) resolves depth and ambiguity ties with `if ref_n <= qry_n`, so the reference wins ties and its consensus becomes the anchor that the merged block inherits (`MergePromise::solve_promise` returns `self.anchor_block`).

This also explains the reported *"the file listed last dominates"* pattern: the last-listed file receives the highest index, hence a lexicographically large name, hence it is usually the reference, hence usually the anchor — so its sequence dominates the merged consensus.

1.3.1 A second, independent defect

`align_with_minimap2_lib_impl` collects its results with `par_bridge()` (line 65). Rayon documents that *"the resulting iterator is not guaranteed to keep the order of the original iterator"* (`rayon-1.12.0/src/iter/par_bridge.rs:26`). `filter_matches` (`pangraph/graph_merging.rs:199`) then applies a *stable* sort by energy and greedily accepts non-overlapping matches, so equal-energy ties are resolved by an order Rayon explicitly does not guarantee. The reported control run was byte-identical, so this is not what was observed — but it is latent nondeterminism in the same code path and should be closed alongside.

1.4 Why this needs fixing

- **It changes the biology.** Block boundaries are where recombination breakpoints are called downstream. A 12% swing in block count driven by argument order propagates directly into biological conclusions.
- **It defeats the guide tree's purpose.** Users pass `--guide-tree` precisely to control merging and obtain comparable results. The current behaviour silently ignores that intent for the query/reference decision.

- **It is non-monotonic and unpredictable.** Because the comparison is lexicographic on decimal strings, behaviour changes discontinuously as block counts cross powers of ten. There is no mental model a user could form to anticipate it.
- **It is a latent-defect generator.** Opaque identifiers are steering an algorithmic decision. Any future change to identifier assignment silently changes scientific output.
- **`-X` discards real signal.** The two directions find measurably different homology (147 vs 146 hits). Currently one is chosen arbitrarily.

1.5 The fix

Attack the point where the identifier leaks into the algorithm: the aligner boundary. Order and name blocks by a hash of their consensus, so that the entire aligner input is a pure function of the block content multiset.

1.5.1 Canonical block naming

```
use std::collections::{BTreeMap, HashMap};
use std::hash::{Hash, Hasher};
use twox_hash::XxHash64;

/// Content-derived naming and ordering for the blocks handed to an aligner.
///
/// Names are fixed-width so that byte-wise (`strcmp`) comparison agrees with the
/// numeric `(consensus_hash, block_id)` order that `canonical_order` uses.
pub struct BlockNames {
  names: BTreeMap<BlockId, String>,
  ids: HashMap<String, BlockId>,
  keys: BTreeMap<BlockId, (u64, BlockId)>,
  order: Vec<BlockId>,
}

/// Order-independent key for a block: hash of its consensus, tie-broken by id.
fn consensus_key(id: BlockId, block: &PangraphBlock) -> (u64, BlockId) {
  let mut hasher = XxHash64::with_seed(0);
  block.consensus().hash(&mut hasher);
  (hasher.finish(), id)
}

impl BlockNames {
  pub fn from_blocks(blocks: &BTreeMap<BlockId, PangraphBlock>) -> Self {
    let keys: BTreeMap<BlockId, (u64, BlockId)> =
      blocks.iter().map(|(&id, b)| (id, consensus_key(id, b))).collect();

    let mut order: Vec<BlockId> = keys.keys().copied().collect();
    order.sort_by_key(|id| keys[id]);

    // 16 hex digits for the hash, 20 decimal digits for the id (usize::MAX has 20).
    // Both zero-padded, so lexicographic order == numeric order at every position.
    let names: BTreeMap<BlockId, String> = keys
      .iter()
      .map(|(&id, &(h, _))| (id, format!("{h:016x}_{:020}", id.0)))
      .collect();

    let ids = names.iter().map(|(&id, n)| (n.clone(), id)).collect();

    Self { names, ids, keys, order }
  }
```

```rust
  /// Blocks in canonical (content-derived) order.
  pub fn canonical_order(&self) -> impl Iterator<Item = BlockId> + '_ {
    self.order.iter().copied()
  }

  pub fn name(&self, id: BlockId) -> &str {
    &self.names[&id]
  }

  /// Recover the `BlockId` an aligner hit refers to. Fallible: an unknown name means
  /// the aligner returned something we did not feed it.
  pub fn id_of(&self, name: &str) -> Result<BlockId, Report> {
    self
      .ids
      .get(name)
      .copied()
      .ok_or_else(|| make_internal_report!("Aligner returned unknown sequence name
'{name}'"))
  }

  /// Order-independent sort key, for canonical tie-breaking downstream.
  pub fn sort_key(&self, id: BlockId) -> (u64, BlockId) {
    self.keys[&id]
  }
}
```

1.5.2 Plugging it into the aligner

```rust
pub fn align_with_minimap2_lib(
  blocks: &BTreeMap<BlockId, PangraphBlock>,
  names: &BlockNames,
  params: &AlignmentArgs,
) -> Result<Vec<Alignment>, Report> {
  // Canonical order, canonical names: nothing here depends on BlockId assignment.
  let (seq_names, seqs): (Vec<&str>, Vec<&str>) = names
    .canonical_order()
    .map(|id| (names.name(id), blocks[&id].consensus().as_str()))
    .unzip();

  align_with_minimap2_lib_impl(&seqs, &seq_names, names, params)
}
```

1.5.3 Deterministic parallel output ordering

Replace the unordered bridge with an indexed parallel iterator. `par_iter` over a slice is an `IndexedParallelIterator`, for which `collect` into a `Vec` preserves input order by construction:

```rust
  let results: Vec<Minimap2Result> = seqs
    .par_iter()
    .zip(names_v.par_iter())
    .map_init(
      || Minimap2Mapper::new(&idx).unwrap(),
      |mapper, (seq, name)| {
        mapper
          .run_map(seq, name)
          .wrap_err_with(|| format!("When aligning sequence '{name}'"))
```

```
      },
    )
    .collect::<Result<Vec<_>, Report>>()?;
```

This is not merely a determinism fix: indexed splitting also gives Rayon better work division than `par_bridge`, which has to serialise pulls from the underlying sequential iterator behind a mutex.

`self_merge`'s other parallel stage, `mergers.into_par_iter()` (`pangraph/graph_merging.rs:145`), is already indexed (`Vec::into_par_iter`) and needs no change.

1.5.4 Canonical downstream tie-breaks

Two places consume the query/reference distinction and must stop privileging the reference.

Energy sorting in `filter_matches`, currently a stable sort whose ties fall through to vector order:

```
  let alns = alns
    .iter()
    .map(|aln| (aln, alignment_energy2(aln, args)))
    .filter(|(_, energy)| *energy < 0.0)
    .sorted_by(|(a, ea), (b, eb)| {
      OrderedFloat(*ea)
        .cmp(&OrderedFloat(*eb))
        .then_with(|| names.sort_key(a.qry.name).cmp(&names.sort_key(b.qry.name)))
        .then_with(|| a.qry.interval.start.cmp(&b.qry.interval.start))
        .then_with(|| names.sort_key(a.reff.name).cmp(&names.sort_key(b.reff.name)))
        .then_with(|| a.reff.interval.start.cmp(&b.reff.interval.start))
    })
    .map(|(aln, _)| aln)
    .collect_vec();
```

Anchor selection in `assign_anchor_block`:

```
fn assign_anchor_block(mergers: &mut [Alignment], graph: &Pangraph, names: &BlockNames)
{
  for m in mergers.iter_mut() {
    let ref_block = &graph.blocks[&m.reff.name];
    let qry_block = &graph.blocks[&m.qry.name];

    let n_of = |b: &PangraphBlock, iv: &Interval| {
      b.consensus()[iv.to_range()].iter().filter(|c| c.0 == b'N').count()
    };

    // Deeper block wins; then fewer ambiguous bases; then a content-derived key.
    // The final arm replaces `ref_n <= qry_n`, which privileged the reference and
    // therefore leaked input order into the choice of surviving consensus.
    let anchor = ref_block
      .depth()
      .cmp(&qry_block.depth())
      .then_with(|| n_of(qry_block, &m.qry.interval).cmp(&n_of(ref_block,
&m.reff.interval)))
      .then_with(|| names.sort_key(m.qry.name).cmp(&names.sort_key(m.reff.name)));

    m.anchor_block = Some(match anchor {
      Ordering::Less => AnchorBlock::Qry,
      _ => AnchorBlock::Ref,
    });
```

```
  }
}
```

1.5.5 Why this is stronger than ordering identifiers by the guide tree

An obvious alternative is to assign singleton identifiers in tree-leaf order rather than FASTA order when `--guide-tree` is given. It does achieve argument-order independence, but it is strictly weaker:

- It only covers the `--guide-tree` path. The neighbour-joining path has its own order dependence: `pair()` uses `Q.argmin()` (`tree/neighbor_joining.rs:64`), which returns the first minimum in row-major order, tie-broken by input position.
- It trades one arbitrary convention for another. `((A,B),C)` and `((B,A),C)` are the same topology but would still give different graphs.
- It has an implementation trap: `PathId.0` is used as a direct index into the FASTA vector (`reconstruct/reconstruct_run.rs:62` together with `build/build_run.rs:44`), so renumbering without permuting the records silently verifies against the wrong sequences.

By contrast, canonicalising the aligner boundary removes the dependence on *all* of these at once. Because `graph_join` is symmetric (`map_merge` over `BTreeMaps`) and `merge_graphs` uses left/right only for debug logging, sibling order in the tree also stops mattering once the aligner input is canonical.

1.6 Problems this could introduce

1.6.1 Hash collisions between identical blocks

This is the sharpest hazard, and naming purely by content hash would be a regression.

Two distinct blocks can legitimately carry an identical consensus — repeated elements, or duplicated regions in the same genome that have not yet merged. If both were given the same name, `skip_seed` (`map.c:81`) would take the `MM_F_NO_DIAG` branch:

```
cmp = strcmp(qname, s->name);
if ((flag&MM_F_NO_DIAG) && cmp == 0 && (int)s->len == qlen) {
    if ((uint32_t)r>>1 == (q->q_pos>>1)) return 1; // avoid the diagonal anchors
    ...
}
```

For two identical sequences the true alignment *is* the diagonal, so every anchor supporting it would be discarded and the pair would never merge — precisely the pair we most want merged. `find_matches` would compound this: it filters `m.qry.name != m.reff.name` to drop self-alignments, which with colliding names would also drop the genuine cross-block hit.

The `{hash}_{id}` suffix resolves this. Distinct blocks always have distinct names, so `cmp == 0` occurs only for a true self-comparison, and `NO_DIAG` retains exactly its intended meaning.

The residual is bounded and benign. When two blocks share a consensus, the direction falls back to `BlockId`, which is still input-order dependent. But the two sequences are identical, so the alignment is symmetric and the anchor choice affects only which *identifier* survives, not the consensus. Downstream, identifiers no longer influence alignment, so the effect is confined to labels in the output JSON. This should be stated in the docs rather than papered over: invariance is guaranteed when consensus sequences are distinct.

A true 64-bit `XxHash64` collision between *different* sequences is a separate matter. At $\sim 10^4$ blocks the birthday probability is $\sim 10^{-11}$, and the consequence is merely a different-but-still-deterministic ordering, not incorrect output, since the identifier recovered from the name is exact. No mitigation needed.

1.6.2 Consequences of the identifier lookup table

Replacing `BlockId::from_str(&paf.q.name)` with a table lookup has a wider blast radius than it first appears:

- **Signature churn.** `Alignment::from_minimap_paf_obj` (`align/minimap2_lib/align_with_minimap2_lib.rs:89`) and `Alignment::from_paf_str` (`align/mmseqs/paf.rs:40`) must both take `&BlockNames`. `find_matches` and `filter_matches` (`pangraph/graph_merging.rs:176`, `:187`) gain the parameter, and `self_merge` constructs the table once per iteration from `graph.blocks`.
- **The mmseqs path breaks silently otherwise.** `PafTsvRecord` deserialises `query: BlockId` and `target: BlockId` directly through serde (`align/mmseqs/paf.rs:15`, `:20`), which only works while names are bare decimal integers. With canonical names this must become a `String` field parsed through `id_of`. Missing this would turn a working backend into a parse error — caught by compilation only if the field type is changed deliberately.
- **Lookups become fallible.** An unknown name currently cannot happen; with a table it can, so the error path needs a real internal error rather than an `unwrap`. This is a strict improvement in diagnosability.
- **Cost is negligible.** The table is `O(n_blocks)` entries (hundreds to low thousands), rebuilt once per `self_merge` iteration. Hashing all consensuses costs one pass over the block sequence set — a few Mbp at `XxHash64` throughput, i.e. milliseconds against multi-second alignment stages.

1.6.3 The mmseqs backend needs the same treatment

`align_with_mmseqs` (`align/mmseqs/align_with_mmseqs.rs:33`) writes its FASTA in `BTreeMap` order under `id.to_string()` names, so it carries the identical defect by a different route. It does not pass `-X`, so it returns both directions; `filter_matches` then accepts one and rejects its mirror as overlapping, with the choice again falling to sort ties. Building `BlockNames` at the `find_matches` level rather than inside the minimap2 backend fixes both backends at once and keeps them consistent.

1.6.4 Downstream fixtures and `pypangraph`

`pypangraph` needs no code change. Its schema (`pypangraph/pangraph_schema.py`) is autogenerated from the Rust types and covers only the serialised graph — `Pangraph`, `PangraphPath`, `PangraphBlock`, `PangraphNode`, `Edit` — with no reference to `Alignment`, `Hit` or the PAF types. `BlockId` remains a `uint`, so the loader, `IndexedCollection` and the integer/string identifier duality are all untouched.

What does need attention is *fixture regeneration*. `packages/pypangraph/tests/data/plasmids.json` is committed data, so it does not move when the build changes — but the moment it is regenerated with a fixed binary, block boundaries, block counts and identifiers all shift, and the hard-coded expectations in `tests/test_graph.py` break:

location	pinned expectation
`test_graph.py:101`	literal block id `"14710008249239879492"`
`test_graph.py:61-63`	137 blocks, 27 core, 10 duplicated
`test_graph.py:88-90`	137×15 matrix, sum 1042

The same caveat applies to `data/test_graph.json` on the Rust side if it is ever rebuilt rather than merely consumed. Neither fixture is regenerated by this change, so nothing breaks on landing; the point is that regeneration must be a deliberate, separate step, with the assertions above updated in the same commit rather than rediscovered as a mysterious test failure later.

Worth recording in `pypangraph`'s changelog regardless: even after the structural guarantee lands, `BlockId` values still derive from `fasta.index`, so downstream code must not assume identifiers are stable across runs with different input order.

1.6.5 Things that are safe

- **Correctness of merging is unaffected.** Which block is anchor changes which valid consensus is retained, not whether the result is valid. Sequence reconstruction and the existing sanity checks are indifferent to the choice.
- **Integration tests do not exercise this path.** `data/test_graph.json` is consumed by the export tests (`itest_export_*.rs`), which read a pre-built graph. They will not shift.
- **No serialised format changes.** `Hit.name` remains a `BlockId`; canonical names exist only for the duration of an alignment call and never reach disk.
- **No performance regression.** `-X` still halves the pair count; the index size and mapping work are unchanged; the parallel stage gets marginally better splitting.

1.6.6 Residual order dependence after the fix

Honesty about what is *not* fixed:

1. Neighbour-joining tie-breaks (`Q.argmin()`) still resolve by input position. With real-valued mash distances exact ties are rare, but this remains and deserves a separate content-derived tie-break.
2. Identical-consensus blocks, as discussed, retain identifier-level (not sequence-level) order dependence.
3. `BlockId` values themselves still derive from `fasta.index`, so the identifiers appearing in output JSON differ between argument orders even when the graph structure is identical. If byte-identical output is wanted, singleton identifiers would also need a content-derived assignment — worth considering, but orthogonal to the correctness issue.

With those caveats, the guarantee becomes: *the graph structure is a pure function of the input sequence set, the tree topology, and the alignment parameters* — independent of argument order and of sibling order within the tree.

1.7 Validation plan

1. Re-run the six-permutation matrix from `tmp/pangraph_order_bug/run.sh`; block counts and block consensus sets must collapse to a single value.
2. Add a regression test asserting that a permuted FASTA argument list yields identical block consensus multisets, both with and without `--guide-tree`.
3. Add a test that a sibling-swapped guide tree (`((A,B),C)` vs `((B,A),C)`) yields identical output.
4. Add a unit test with two blocks sharing an identical consensus, asserting they still merge — this is the collision hazard, and it would fail under naive content-hash naming.
5. Confirm the mmseqs backend agrees with minimap2 on direction selection.
6. Confirm no fixture is regenerated as a side effect: `data/test_graph.json` and `packages/pypangraph/tests/data/plasmids.json` must be byte-identical after the change, so that the test suites verify the fix rather than absorb it.